USN NUMBER: 1RUB25CSE0002
NAME: ARUNDATHI B

Ex No: 1 **Date: 27-02-26**	**Building and Training a 2 Layer Neural Network for Binary Classification**

Objective:

To build a 2 Layer Neural Network to recognize car vs bike using Gradient descent implementation.

Descriptions:

Binary classification is the task of classifying elements of a given set into two groups. A 2-layer neural network can be used for binary classification. We have an input image $x$and the output $y$is a label to recognize the image.

1 means a **car** is in the image, 0 means a **bike** is in the image.A 2-layer neural network is a supervised learning algorithm that we use when labels are either 0 or 1, which is known as a Binary Classification Problem. An input feature vector Xcorresponds to an image that we want to recognize as either a car (1) or a bike (0). The algorithm outputs a prediction, which is an estimate of y.

Unlike Logistic Regression, a 2-layer neural network contains one hidden layer. The architecture is:

Input Layer → Hidden Layer (ReLU activation) → Output Layer (Sigmoid activation)

The hidden layer allows the network to learn nonlinear relationships between input features and the output. The final layer uses the sigmoid function to produce a probability value between 0 and 1.

If we initialize the weights to small random values (not zeros), different neurons in the hidden layer learn different features. This is important because if all weights are initialized to zero, every neuron in the hidden layer would learn the same thing due to symmetry, and the network would not learn properly.

During training:

- Forward propagation computes predictions.
- The cost function measures error.
- Backpropagation computes gradients.
- Parameters are updated using gradient descent.

USN NUMBER: 1RUB25CSE0002
NAME: ARUNDATHI B

The output of the model is:

$$A^{[2]} = \sigma(W^{[2]}A^{[1]} + b^{[2]})$$

where:

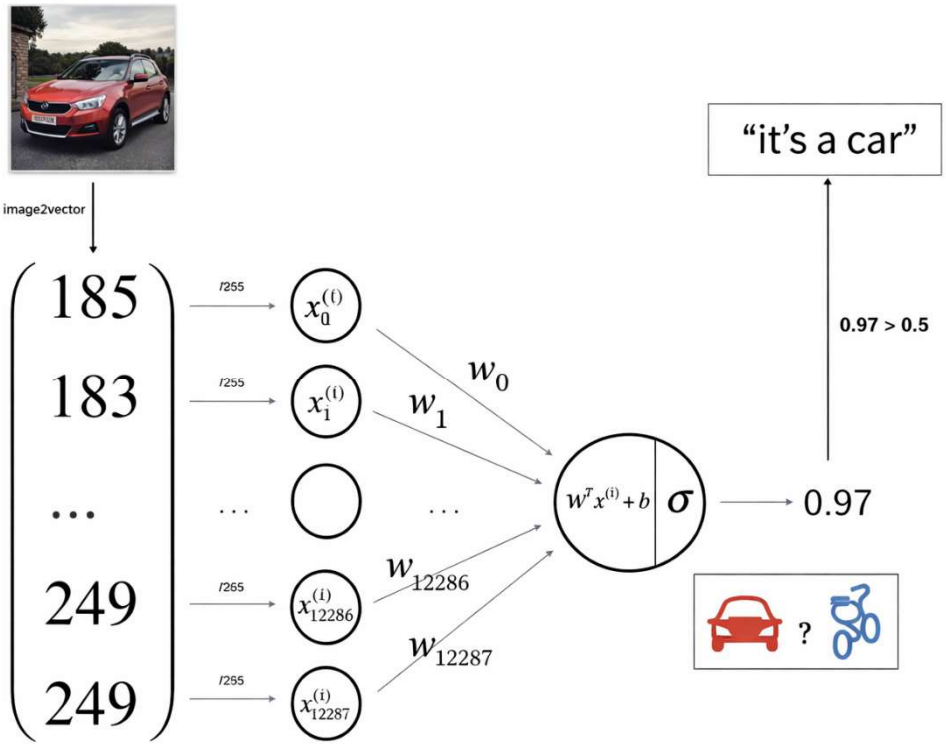
$$A^{[1]} = ReLU(W^{[1]}X + b^{[1]})$$

The final output is a probability:

- If output > 0.5 → Car (1)
- If output ≤ 0.5 → Bike (0)

This model extends logistic regression by adding a hidden layer, enabling it to learn more complex decision boundaries.

Model:



USN NUMBER: 1RUB25CSE0002
NAME: ARUNDATHI B

Building the parts of algorithm

The main steps for building the 2-Layer Neural Network are:

1. Define the model structure - In this implementation, the model structure consists of one hidden layer and one output layer. The input layer has 12288 units (64 × 64 × 3 flattened image pixels). The hidden layer contains a chosen number of neurons (for example, 10 neurons) with ReLU activation. The output layer contains 1 neuron with Sigmoid activation for binary classification (car = 1, bike = 0).

2. Initialize the model's parameters - The parameters W^1, b^1, W^2, and b^2 are initialized before training. The weight matrices are initialized with small random values, and the bias vectors are initialized to zeros. Random initialization ensures that neurons in the hidden layer learn different features and prevents symmetry during learning.

3. Loop

   Calculate current loss (forward propagation) - Forward propagation is performed by first computing $Z^1 = W^1X + b^1$ and applying ReLU activation to obtain A^1. Then compute $Z^2 = W^2A^1 + b^2$ and apply the Sigmoid function to obtain A^2, which represents the predicted probability that the image is a car. The binary cross-entropy cost function is used to measure the error between predictions and true labels.

   Calculate current gradient (backward propagation) - Backward propagation computes the gradients of the cost with respect to all parameters. First, compute the error at the output layer, then propagate it backward through the hidden layer using the derivative of ReLU. Gradients dW^1, db^1, dW^2, and db^2 are calculated.

   Update parameters (gradient descent) - All parameters are updated using gradient descent by subtracting the learning rate multiplied by their corresponding gradients. This process is repeated for a fixed number of iterations until the cost decreases and the model learns meaningful patterns.

   After training, the final output A^2 is interpreted as follows: if $A^2 > 0.5$, the image is classified as Car (1); otherwise, it is classified as Bike (0).

GitHub Link: https://github.com/karunaarundhati/Deepl_learning_Assignment